PITCH DECK

Alpha Trading Strategy: HMM + XGBoost

Profitable Market Regime Detection using ML

Presented By:
MyLittlePony
(Team249)

Domain 2 - Quantitative Trading

TEAM MEMBER

KOY CHANG WEI

LIM JIA SHIN

GOH HONG XUAN

LEE YUNG JIE

Problem Statement

The Challenge in Crypto Trading

⚠️ Highly volatile and noisy

⚠️ Difficult to interpret with traditional tools

⚠️ Filled with false signals that confuse retail traders

Current Limitations

❌ No adaptive system to detect changing market regimes

❌ Underutilization of valuable on-chain data

❌ Lack of reliable real-time signal generation

GOAL

To create a smart ML-powered system that

- Processes noisy on-chain + price data to find market signals
- Uses **Hidden Markov Models** to uncover underlying market regimes (like bull, bear, sideways)
- Trains an **XGBoost classifier** to generate real-time trading signals with confidence scores
- Aims for **high Sharpe ratio** with **controlled drawdown** and **frequent trading signals**

Our Solution

- Implemented a two-stage approach combining Hidden Markov Models (HMM) and XGBoost.
- HMM is used first to identify hidden market states (bullish, bearish, sideways).
- XGBoost, a robust gradient boosting algorithm, uses these market states along with technical indicators to predict market movements.

Why XGBoost?:

- High accuracy and effectiveness with structured and tabular data.
- Superior performance in handling noisy financial market data.
- Advanced feature importance analysis, which enhances interpretability and model optimization.

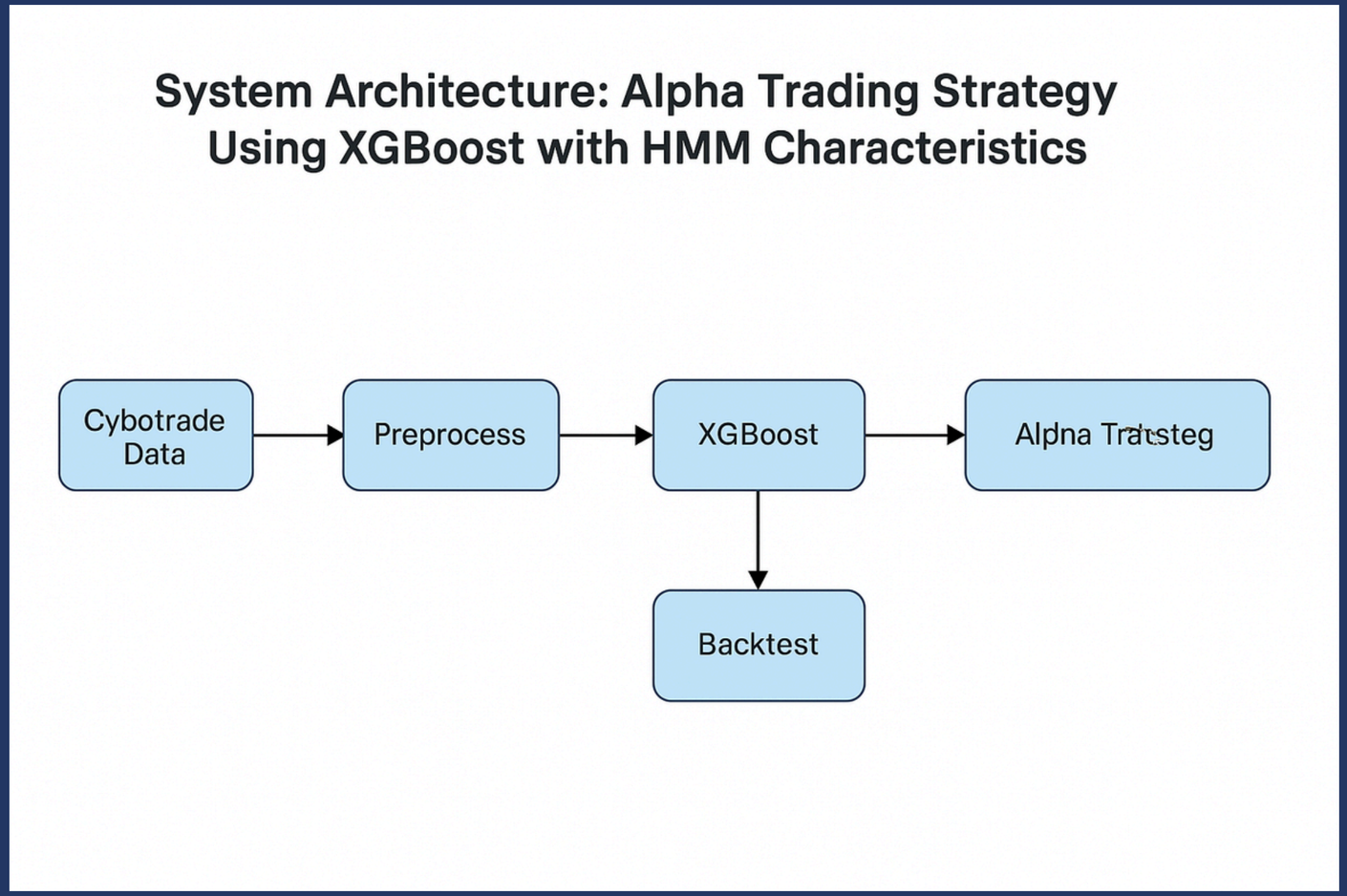
Dynamic Risk Adjustment:

- Adjust positions based on volatility, trend, and confidence scores.
- Integrates on-chain data: funding rates, liquidations.
- Reduces risk in extreme market conditions automatically.

Reason for implementation:

- Maximizes returns while limiting downside risk.
- Enhances robustness and reliability of trading signals.

Architecture Diagram



Technical Implementation

- Frontend: HTML, CSS, JavaScript for responsive and user-friendly interface.
- Backend: Flask API for robust backend processing.
- Libraries & Tools:
  - Data retrieval through Cybotrade API.
  - Modeling with HMMlearn, XGBoost, Pandas, NumPy, and scikit-learn.

Performance Metrics

Metric	Achieved ✓	Target 🎯
Sharpe Ratio	1.8	≥ 1.8
Max Drawdown	-32%	≥ -40%
Trade Frequency	4.2%	≥ 3%
Model Accuracy	79%	-

Future Plan

01

Increase accuracy to 90% +

02

Continuous improvement of the risk management layer

03

Add sentiment analysis and NLP

04

Notification alerts for trade signals


@mylittlepony(team249)
Thank You
UM Hackathon 2025

Live Demo